

Drop Radar: project brief

Kevin Pham · UW undergraduate · prepared for a design-review conversation with Prof. Michael Ernst

What it is

Drop Radar is a service that watches employers' job boards and alerts students to new internships within minutes. I built it because I was tired of refreshing job sites. Now I want to extend it to the part that actually gets people interviews: reaching a real person who can refer you, without burning out the people who help.

What exists today (live since early October)

- **Over 600 career sources** polled politely (conditional requests, per-host limits, `robots.txt`), about **12,000 postings** tracked, phone push alerts, and a private "Career library" of versioned, approved facts.
- **About 9,500 lines of Python** (asyncio, FastAPI, SQLite in WAL mode) and **6,000 lines of TypeScript** (React PWA), with about **520 backend tests and 74 browser end-to-end tests**.
- **Two users** so far (me and a friend). Started 2026-09-17.
- **Most of the code was written with AI coding assistants** (Claude and Codex), which is one reason I want an experienced software engineer's view of how to verify it.

Design decisions I would like critiqued

1. **Seed, then diff.** A source's first poll is stored as silent backfill, so only postings that appear later can alert.
2. **Claim before send.** An alert row is claimed in the database before the push goes out, so a crash or a concurrent sweep cannot double-send.
3. **One predicate for feed and phone.** What a user sees and what alerts their phone use the same function, so they cannot disagree.
4. **A bitmap index for list queries.** Moved lists from 0.4–1.4 s to 5–30 ms on a small VM.
5. **Immutable revisions with optimistic concurrency** for the Career library; changed supporting facts flag answers for re-review.

What I plan to build next (not built yet)

A "referral layer": for a job, show who can get you in (people you have emailed, UW alumni at the company, alumni who opted in as referrers), draft an honest note using only approved facts, send it only after the student taps, and track outcomes. Alumni set a monthly limit on how many asks they accept, counted across all students, so a few people do not get flooded. Private data (email headers, career facts) goes only to one model provider; public data may go to cheaper ones.

Where your expertise would help most

1. **Verifying LLM-written code** that handles private data and login tokens. What would you require before trusting it?
2. **Testing "never send twice, never send without a tap,"** including crashes mid-send. My crash test with a workflow library showed that a crash after the side effect but before the checkpoint re-runs the step.
3. **Enforcing a data-flow rule** (private data to one provider only) so a later change cannot break it.
4. **What I can honestly conclude** from a six-week pilot with about 30 students.
5. **Scope:** with six weeks, what would you build first and cut?

What I have not verified

No load or multi-user testing beyond two accounts. A ten-case comparison of cheaper models against my default is still open. One production restart exceeded the 45-second shutdown timeout. The referral layer is a plan, not code. I can walk through the live site, the architecture and the code at the meeting.